

DEV-592: Research Redis Sentinel vs Redis Cluster for high availability (HA)

Department: DevOps

Prepared by: Thanusha Bai V

Date: 17-sept-2026

Task: DEV-587-Redis Security Hardening

1. Executive Summary

This report compares Redis Sentinel and Redis Cluster as high-availability (HA) solutions for Glynac. The two solve **different problems**: Sentinel provides automatic failover for a **single dataset** that fits on one node, while Cluster adds **horizontal scaling** by sharding data across multiple masters. Sentinel is simpler, cheaper, and sufficient when Glynac's Redis data fits comfortably on one machine. Cluster becomes necessary when data volume or write throughput outgrows a single node. The report provides a decision framework and a recommended evolution path: start with Sentinel, migrate to Cluster only when capacity demands it.

2. What Each Solution Actually Does

2.1 Redis Sentinel – High Availability for One Dataset

Sentinel is a set of independent monitoring processes that watch a single master and its replicas. When a quorum of Sentinels agrees the master is down, they **elect a leader and promote a replica** to become the new master, then notify clients to reconnect.

What Sentinel provides:

- Automatic failover (typically 10–30 seconds)
- Read scaling via replicas
- Transparent client redirection (Sentinel-aware clients)

What Sentinel does NOT provide:

- Data sharding – every key still lives on one master
- Write scaling – all writes still go to the single master
- Capacity beyond one node's memory/CPU

Minimum production topology: 1 master + 2 replicas + 3 Sentinel nodes (spread across separate failure domains).

2.2 Redis Cluster – Horizontal Scaling + HA

Cluster partitions the keyspace across **16,384 hash slots**, distributing them among multiple masters. Each master owns a slot range and has its own replica. If a master fails, its replica is promoted automatically – **no separate Sentinel processes needed**.

What Cluster provides:

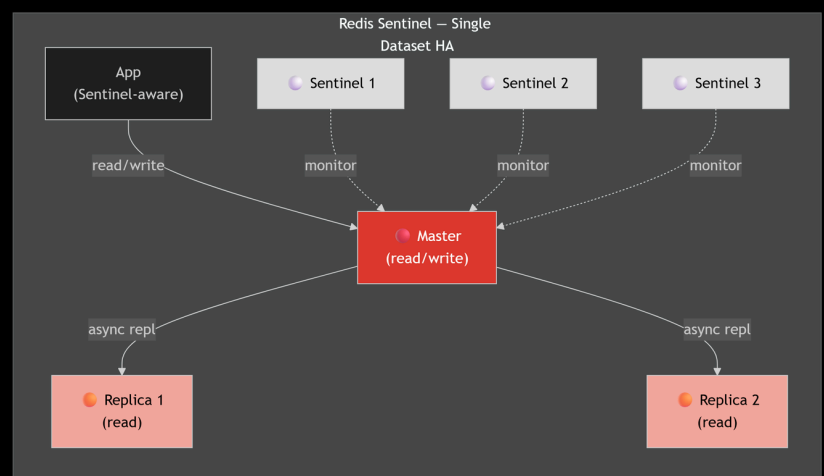
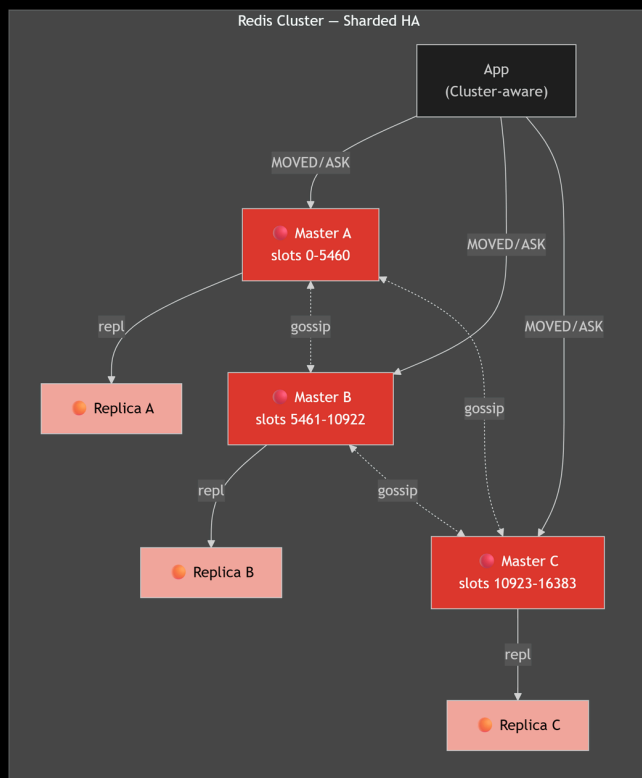
- Horizontal scaling – add masters to grow capacity and throughput
- Built-in failover per shard
- Data partitioned across nodes (no single-node memory limit)

What Cluster costs:

- Higher operational complexity (slot management, resharding)
- Cluster-aware clients required (handle MOVED/ASK redirects)
- Multi-key operations restricted to the same hash slot
- No cross-slot transactions or Lua scripts

Minimum production topology: 3 masters + 3 replicas (6 nodes total).

3. Architecture Diagram



Legend: Solid arrow = data flow / replication. Dashed arrow = monitoring / control. Red = master, light red = replica, gray = Sentinel.

4. Side-by-Side Comparison

Dimension	Sentinel	Cluster
Primary purpose	HA for single dataset	Horizontal scaling + HA
Data sharding	No – one master holds all keys	Yes – 16,384 hash slots across masters
Write scaling	No – single master bottleneck	Yes – writes spread across masters
Capacity limit	Single node memory	Combined memory of all masters
Failover mechanism	External Sentinel processes vote	Masters monitor each other; replica promoted
Failover time	~10-30 seconds	Similar (depends on cluster-node-timeout)
Minimum nodes	3 (1 master + 1 replica + 3 Sentinels)	6 (3 masters + 3 replicas)
Client requirements	Sentinel-aware	Cluster-aware (MOVED/ASK handling)
Multi-key operations	Unlimited	Same slot only (hash tags required)
Transactions across keys	Supported	Restricted
Operational complexity	Lower	Higher
External proxy needed?	Usually no	No, but NodePort/LB per node often required

5. Decision Framework for Glynac

Ask these questions in order:

Q1: Does the entire Redis dataset fit comfortably on one machine?

- **Yes** → Sentinel is sufficient. Don't add Cluster complexity.
- **No** (dataset > single-node memory, or approaching 70%+ memory consistently) → Cluster is required.

Q2: Is write throughput saturating a single master?

- **No** → Sentinel is fine.
- **Yes** (single master CPU consistently >75%, write QPS maxed) → Cluster.

Q3: Does the application use multi-key operations, transactions, or Lua scripts heavily?

- **Yes** → Prefer Sentinel; Cluster's cross-slot restrictions require application changes.
- **No** → Cluster is viable.

Q4: What's Glynac's team operational capacity?

- Small team, limited Redis ops experience → Sentinel first.
- Mature SRE capability, ready for stateful distributed systems → Cluster acceptable.

Decision Summary

Glynac Scenario	Recommendation
Cache layer, <8GB data, read-heavy	Sentinel
Sessions/queues, single-node capacity	Sentinel
Data >64GB or write throughput > single node	Cluster
Multi-tenant cache with growing capacity needs	Cluster (long-term)
Small team, limited HA ops experience	Sentinel

6. Recommended Evolution Path

Phase 1 (Current): Deploy **Sentinel** with 1 master + 2 replicas + 3 Sentinel nodes. This delivers automatic failover without the complexity of sharding. Most Glynac workloads will fit here.

Phase 2 (Trigger-based migration to Cluster): Migrate when **any** of these triggers occur:

- Dataset consistently exceeds 70% of single-node memory
- Write throughput saturates a single master
- Weekly data growth exceeds 10% and capacity planning shows <6 months headroom
- Multiple teams need isolated key spaces that exceed one node

Migration approach: Build the Cluster in parallel, use dual-write or a sync tool to migrate data, then cut traffic over.

7. Key Trade-offs to Document

1. **Sentinel does not solve capacity.** Adding replicas increases read capacity and redundancy, not write capacity or memory.
 2. **Cluster is not "more advanced Sentinel."** It's a different architecture with real application constraints (cross-slot limits).
 3. **Sentinel + Cluster is not a valid combination.** Cluster has built-in failover; you don't run Sentinel on top of it.
 4. **Client code matters.** Sentinel-aware clients discover the master; Cluster-aware clients handle redirects. Glynac's application stack must support whichever is chosen.
 5. **Network topology matters.** Cluster requires clients to reach each node directly; reverse proxies break Cluster protocol.
-

8. Summary

Area	Finding
Sentinel solves	Automatic failover for single dataset
Cluster solves	Horizontal scaling + per-shard failover
Sentinel limit	Single-node capacity ceiling
Cluster cost	Operational complexity + client changes
Recommended starting point	Sentinel (1 master + 2 replicas + 3 Sentinels)
Migration trigger	Data/throughput outgrows single node
Glynac decision rule	Sentinel first; Cluster only when capacity demands it

9. Recommendations for Glynac

1. **Start with Sentinel** for all current Redis deployments unless data already exceeds single-node capacity.
2. **Define migration triggers** (memory %, write QPS, growth rate) and revisit quarterly.
3. **Ensure client libraries** support Sentinel discovery; test failover behavior in staging.
4. **Document key design** now to avoid Cluster migration pain later (use hash tags for related keys).
5. **Avoid Cluster prematurely** – the operational cost is real, and Sentinel covers most HA needs.

This research provides the decision framework for Glynac's Redis HA architecture. Final selection depends on current dataset size and growth projections.

END OF REPORT